```
    }
  } finally {

readLock.unlock();
            #3
  }
}
```

That leads to 4 simple rules to handle our locks, shown in table 7.4.

If any (other) thread is holding a read lock, we can acquire another read lock, but need to wait for a write lock till all the read locks are released.

If any thread is holding a write lock, all other threads need to wait to acquire either a write or a read lock.

Table 7.4 Combinations of locks requested and held

Lock Requested/Held	Read	Write
Read	Allowed	Need to wait
Write	Need to wait	Need to wait

So, the advantage of actually distinguishing between read and write locks for our cache is that all calls to `get` can be executed in parallel, and only calls to set will block. Hadn't we made this distinction, access to the cache would have resulted entirely synchronous, with only one thread at the time able to check the cache. This would have introduced an